SIMATS ENGINEERING

ASSESSMENT TOOL 1 - High (INDUSTRY PROBLEM SOLVING TASKS)

COURSE CODE: CSA09 **COURSE NAME: Programming in Java**

COURSE OUTCOME COVERED

CO4: *Design and develop GUI-based user interfaces using Abstract Window Toolkit, Swing, and Applets by implementing event handling, layout managers, AWT controls, and menus. (BL3,BL5)*

Assessment Tool Weightage: 10%

QUESTIONS: 1 **Total Marks: 50**

ANNEXURE A

INDUSTRY PROBLEM SOLVING TASKS

Code of Conduct:

I Harini G/192524011 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
HariniGopi

A

INDUSTRY PROBLEM SOLVING TASKS

1. Railway Reservation Interface (BTL: BL5 – Evaluate)

Design a Swing-based GUI application for railway ticket booking, including features such as searching trains, booking tickets, and canceling reservations. Use components like combo boxes, tables, and buttons for interaction. Implement event listeners to handle booking and cancellation processes. Validate user inputs such as travel dates and seat availability. Evaluate layout manager choices for organizing multiple UI elements. Analyze system scalability for handling high user traffic and propose improvements.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

SOLUTION:
JAVA CODE:

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.*;

public class RailwayReservation extends JFrame implements ActionListener {

    JComboBox<String> fromBox, toBox, trainBox;
    JTextField dateField, seatField;
    JButton searchButton, bookButton, cancelButton;
    JTable table;
    DefaultTableModel model;

    int availableSeats = 50;

    RailwayReservation() {

        setTitle("Railway Reservation System");
        setSize(750, 500);
        setLayout(new BorderLayout());
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // Top panel
        JPanel topPanel = new JPanel(new GridLayout(3, 4, 10, 10));

        fromBox = new JComboBox<String>(
                new String[]{"Chennai", "Bangalore", "Coimbatore", "Madurai"});

        toBox = new JComboBox<String>(
                new String[]{"Chennai", "Bangalore", "Coimbatore", "Madurai"});

        trainBox = new JComboBox<String>(
                new String[]{"Chennai Express", "Pandian Express",
                        "Coimbatore Express"});

        dateField = new JTextField();
        seatField = new JTextField();

        searchButton = new JButton("Search Train");
        bookButton = new JButton("Book Ticket");
        cancelButton = new JButton("Cancel Reservation");
```

```
topPanel.add(new JLabel("From:"));
topPanel.add(fromBox);

topPanel.add(new JLabel("To:"));
topPanel.add(toBox);

topPanel.add(new JLabel("Train:"));
topPanel.add(trainBox);

topPanel.add(new JLabel("Travel Date (DD-MM-YYYY):"));
topPanel.add(dateField);

topPanel.add(new JLabel("Number of Seats:"));
topPanel.add(seatField);

topPanel.add(searchButton);
topPanel.add(bookButton);

add(topPanel, BorderLayout.NORTH);

// Table
model = new DefaultTableModel();
model.addColumn("Train");
model.addColumn("From");
model.addColumn("To");
model.addColumn("Date");
model.addColumn("Seats");

table = new JTable(model);

add(new JScrollPane(table), BorderLayout.CENTER);

// Bottom panel
JPanel bottomPanel = new JPanel();
bottomPanel.add(cancelButton);

add(bottomPanel, BorderLayout.SOUTH);

// Event listeners
searchButton.addActionListener(this);
bookButton.addActionListener(this);
cancelButton.addActionListener(this);

setVisible(true);
```

```
}

public void actionPerformed(ActionEvent e) {

    // Search button
    if (e.getSource() == searchButton) {

        String from = fromBox.getSelectedItem().toString();
        String to = toBox.getSelectedItem().toString();

        if (from.equals(to)) {
            JOptionPane.showMessageDialog(this,
                    "From and To stations cannot be same!");
            return;
        }

        JOptionPane.showMessageDialog(this,
                "Train found!\nAvailable Seats: " + availableSeats);
    }

    // Book button
    if (e.getSource() == bookButton) {

        try {
            String from = fromBox.getSelectedItem().toString();
            String to = toBox.getSelectedItem().toString();
            String train = trainBox.getSelectedItem().toString();
            String date = dateField.getText();

            if (from.equals(to)) {
                throw new IllegalArgumentException(
                        "From and To stations cannot be same.");
            }

            if (date.isEmpty()) {
                throw new IllegalArgumentException(
                        "Please enter travel date.");
            }

            int seats = Integer.parseInt(seatField.getText());

            if (seats <= 0) {
                throw new IllegalArgumentException(
                        "Number of seats must be greater than 0.");
```

```
        }

        if (seats > availableSeats) {
            JOptionPane.showMessageDialog(this,
                    "Only " + availableSeats +
                    " seats are available.");
            return;
        }

        availableSeats = availableSeats - seats;

        model.addRow(new Object[]{
                train, from, to, date, seats
        });

        JOptionPane.showMessageDialog(this,
                "Ticket booked successfully!\n" +
                "Remaining seats: " + availableSeats);

    } catch (NumberFormatException ex) {

        JOptionPane.showMessageDialog(this,
                "Please enter a valid number of seats.");

    } catch (IllegalArgumentException ex) {

        JOptionPane.showMessageDialog(this,
                ex.getMessage());
    }
}

// Cancel button
if (e.getSource() == cancelButton) {

    int row = table.getSelectedRow();

    if (row == -1) {

        JOptionPane.showMessageDialog(this,
                "Please select a reservation to cancel.");

    } else {

        int seats = Integer.parseInt(
```

```
                    model.getValueAt(row, 4).toString());

            availableSeats = availableSeats + seats;

            model.removeRow(row);

            JOptionPane.showMessageDialog(this,
                "Reservation cancelled successfully!\n" +
                "Available seats: " + availableSeats);
        }
    }
}

public static void main(String[] args) {

    new RailwayReservation();
}
}
```

OUTPUT:

